



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 06: Emergence (cap 12)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

25 de Agosto, 2026

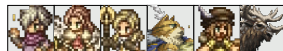
Proyecto

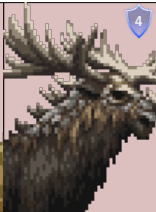
Octopath Traveler

Octopath Traveler es una serie de videojuegos del género JRPG desarrollados por Square Enix junto a Acquire. La saga consiste actualmente de 3 juegos, donde destaca por su sistema de turnos y estilo visual.

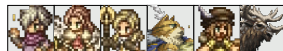


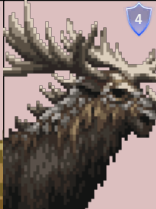
Sistema de turnos



 800 HP Chubby Cait	 1440 HP Demon Deer			
1 BP  1548 HP 257 SP Ophilia	1 BP  2000 HP 135 SP Tressa	1 BP  1376 HP 173 SP Therion	1 BP  1548 HP 103 SP H'aanit	

Sistema de turnos



 234 HP 566 HP Chubby Cait	 1440 HP 1440 HP Demon Deer			
1 BP  1548 HP 257 SP Ophilia	1 BP  2000 HP 135 SP Tressa	1 BP  1376 HP 173 SP Therion	1 BP  1548 HP 103 SP H'aanit	

Boost Points

Los viajeros pueden utilizar Boost Points para amplificar el efecto de su acción.
¿Cómo funcionan estos?

Boost Points

Los viajeros pueden utilizar Boost Points para amplificar el efecto de su acción.
¿Cómo funcionan estos?

- 1 En los ataques básicos, permiten realizar tantos ataques adicionales como Boost Points se utilicen

Boost Points

Los viajeros pueden utilizar Boost Points para amplificar el efecto de su acción.
¿Cómo funcionan estos?

- 1 En los ataques básicos, permiten realizar tantos ataques adicionales como Boost Points se utilicen
- 2 En las habilidades, permite potenciar su efecto, aumentar su duración o hasta cambiar o agregar efectos

Boost Points

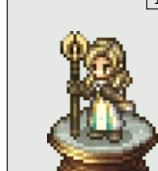
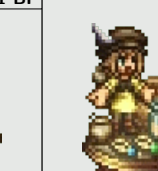
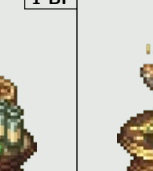
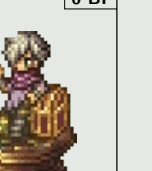
Los viajeros pueden utilizar Boost Points para amplificar el efecto de su acción.
¿Cómo funcionan estos?

- 1 En los ataques básicos, permiten realizar tantos ataques adicionales como Boost Points se utilicen
- 2 En las habilidades, permite potenciar su efecto, aumentar su duración o hasta cambiar o agregar efectos

Todo viajero puede tener un máximo de 5 Boost Points. Al final de cada ronda, todo viajero vivo que no haya usado **BP**, ganará 1 **BP**.

Boost Points



				
332 HP	1440 HP			
Chubby Cait	Demon Deer			
1 BP	1 BP	0 BP	1 BP	
				
1548 HP	2000 HP	1376 HP	1548 HP	
257 SP	135 SP	173 SP	103 SP	
Ophilia	Tressa	Therion	H'aanit	

Debilidades

Las bestias cuentan con una lista de armas o tipos elementales a los que son débiles.

Debilidades

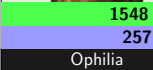
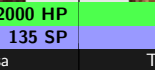
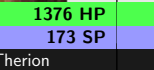
Las bestias cuentan con una lista de armas o tipos elementales a los que son débiles.



- ☞ Sword
- ☞ Axe
- ☞ Dagger
- ☞ Stave

Debilidades



 WEAK 351 HP				
449 HP	1440 HP			
Chubby Cait	Demon Deer			
1 BP	1 BP	1 BP	1 BP	
				
1548 HP	2000 HP	1376 HP	1548 HP	
257 SP	135 SP	173 SP	103 SP	
Ophilia	Tressa	Therion	H'aanit	

Breaking Point

Es un estado que las bestias alcanzan una vez sus Shields llegan a 0. ¿Cómo funciona el Breaking Point?

Breaking Point

Es un estado que las bestias alcanzan una vez sus Shields llegan a 0. ¿Cómo funciona el Breaking Point?

- 1 Una bestia en Breaking Point no podrá jugar su turno de la ronda actual y la siguiente

Breaking Point

Es un estado que las bestias alcanzan una vez sus Shields llegan a 0. ¿Cómo funciona el Breaking Point?

- 1 Una bestia en Breaking Point no podrá jugar su turno de la ronda actual y la siguiente
- 2 Estas reciben más daño por ataque

Breaking Point

Es un estado que las bestias alcanzan una vez sus Shields llegan a 0. ¿Cómo funciona el Breaking Point?

- 1 Una bestia en Breaking Point no podrá jugar su turno de la ronda actual y la siguiente
- 2 Estas reciben más daño por ataque
- 3 Una vez finalizadas las 2 rondas, la bestia saldrá del estado recuperando todos sus Shields y será la primera en atacar en la ronda

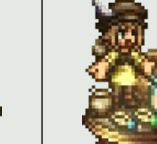
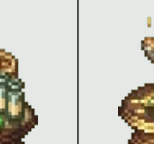
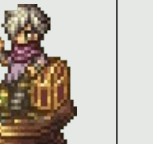
Breaking Point

Es un estado que las bestias alcanzan una vez sus Shields llegan a 0. ¿Cómo funciona el Breaking Point?

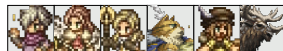
- 1 Una bestia en Breaking Point no podrá jugar su turno de la ronda actual y la siguiente
- 2 Estas reciben más daño por ataque
- 3 Una vez finalizadas las 2 rondas, la bestia saldrá del estado recuperando todos sus Shields y será la primera en atacar en la ronda

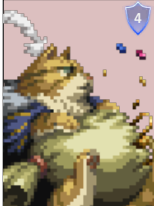
Breaking Point



 WEAK 351 HP BREAK				
449 HP	1440 HP			
Chubby Cait	Demon Deer			
1 BP	1 BP	1 BP	1 BP	1 BP
				
1548 HP	2000 HP	1376 HP	1548 HP	
257 SP	135 SP	173 SP	103 SP	
Ophilia	Tressa	Therion	H'aanit	

Breaking Point



 4	 4			
449 HP	1440 HP			
Chubby Cait	Demon Deer			
3 BP 	3 BP 	3 BP 	3 BP 	3 BP
1548 HP	2000 HP	1376 HP	1548 HP	
257 SP	135 SP	173 SP	103 SP	
Ophilia	Tressa	Therion	H'aanit	

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las bestias tienen distintas debilidades

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las bestias tienen distintas debilidades
- 2 Atacar a estas debilidades pueden “romper” a la bestia y alterar la cola de turnos

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las bestias tienen distintas debilidades
- 2 Atacar a estas debilidades pueden “romper” a la bestia y alterar la cola de turnos
- 3 Hay distintos tipos de habilidades

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las bestias tienen distintas debilidades
- 2 Atacar a estas debilidades pueden “romper” a la bestia y alterar la cola de turnos
- 3 Hay distintos tipos de habilidades
- 4 Las habilidades pueden tener distintos efectos

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las bestias tienen distintas debilidades
- 2 Atacar a estas debilidades pueden “romper” a la bestia y alterar la cola de turnos
- 3 Hay distintos tipos de habilidades
- 4 Las habilidades pueden tener distintos efectos
- 5 Sus efectos pueden cambiar por el uso de mecánicas

Desafíos del proyecto

El juego cuenta con distintos elementos que pueden afectar el flujo del combate y la cola de turnos de los personajes. En particular, el proyecto cuenta con los siguientes desafíos:

- 1 Las bestias tienen distintas debilidades
- 2 Atacar a estas debilidades pueden “romper” a la bestia y alterar la cola de turnos
- 3 Hay distintos tipos de habilidades
- 4 Las habilidades pueden tener distintos efectos
- 5 Sus efectos pueden cambiar por el uso de mecánicas

Pregunta importante: ¿Como diseñamos nuestro código para que agregar nuevos tipos de habilidades y efectos sea simple?

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

- 1 Se deben implementar la debilidades de las bestias y el `Breaking Point`

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

- 1 Se deben implementar la debilidades de las bestias y el Breaking Point
- 2 Deben implementar las acciones Usar Habilidad y Defender

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

- 1 Se deben implementar la debilidades de las bestias y el Breaking Point
- 2 Deben implementar las acciones Usar Habilidad y Defender
- 3 Deben implementar habilidades con los efectos Daño, Recuperación, Revivir y con efecto en la cola de turnos

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

- 1 Se deben implementar la debilidades de las bestias y el Breaking Point
- 2 Deben implementar las acciones Usar Habilidad y Defender
- 3 Deben implementar habilidades con los efectos Daño, Recuperación, Revivir y con efecto en la cola de turnos
- 4 Deben implementar habilidades pasivas que alteran los stats base de los personajes

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

- 1 Se deben implementar la debilidades de las bestias y el Breaking Point
- 2 Deben implementar las acciones Usar Habilidad y Defender
- 3 Deben implementar habilidades con los efectos Daño, Recuperación, Revivir y con efecto en la cola de turnos
- 4 Deben implementar habilidades pasivas que alteran los stats base de los personajes

Los principales desafíos serán diseñar el sistema de cola de turnos y las habilidades de tal forma que implementar nuevas habilidades sea sencillo.

Desafíos de la E2

Para esta entrega, deberán implementar el sistema de debilidades, dos acciones y varios tipos de efectos para las habilidades. En particular:

- 1 Se deben implementar la debilidades de las bestias y el `Breaking Point`
- 2 Deben implementar las acciones `Usar Habilidad` y `Defender`
- 3 Deben implementar habilidades con los efectos `Daño`, `Recuperación`, `Revivir` y con efecto en la cola de turnos
- 4 Deben implementar habilidades pasivas que alteran los stats base de los personajes

Los principales desafíos serán diseñar el sistema de cola de turnos y las habilidades de tal forma que implementar nuevas habilidades sea sencillo.

En esta entrega, evaluaremos los capítulos 2, 3, 6 y 10 de Clean Code, así como el patrón Modelo-Vista-Controlador.

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño
- 2 Deben agregar el sistema de Boost Points en ataques básicos y habilidades

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño
- 2 Deben agregar el sistema de Boost Points en ataques básicos y habilidades
- 3 Deben implementar el uso de habilidades divinas

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño
- 2 Deben agregar el sistema de Boost Points en ataques básicos y habilidades
- 3 Deben implementar el uso de habilidades divinas
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño
- 2 Deben agregar el sistema de Boost Points en ataques básicos y habilidades
- 3 Deben implementar el uso de habilidades divinas
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

El principal desafío será crear habilidades tal que sus efectos pueden cambiar por el uso de Boost Points sin repetir código.

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño
- 2 Deben agregar el sistema de Boost Points en ataques básicos y habilidades
- 3 Deben implementar el uso de habilidades divinas
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

El principal desafío será crear habilidades tal que sus efectos pueden cambiar por el uso de Boost Points sin repetir código.

Se pondrá a prueba que sus códigos permitan cambiar de una vista de consola a una vista de interfaz gráfica de forma sencilla y poco disruptiva.

Desafíos de la E3

Para esta entrega, deberán implementar habilidades que tienen más de un hit, que otorgan buffs y debuffs y el uso de Boost Points. En particular:

- 1 Deben agregar efectos que afectan al cálculo del daño
- 2 Deben agregar el sistema de Boost Points en ataques básicos y habilidades
- 3 Deben implementar el uso de habilidades divinas
- 4 Deben implementar una interfaz gráfica que nosotros les entregaremos

El principal desafío será crear habilidades tal que sus efectos pueden cambiar por el uso de Boost Points sin repetir código.

Se pondrá a prueba que sus códigos permitan cambiar de una vista de consola a una vista de interfaz gráfica de forma sencilla y poco disruptiva.

En esta entrega, evaluaremos todos los capítulos de Clean Code (que vemos en clases) y el patrón Modelo-Vista-Controlador.

Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

- 1 Deben implementar habilidades con nuevos efectos de estado

Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

- 1 Deben implementar habilidades con nuevos efectos de estado
- 2 Deben implementar más habilidades con efectos especiales

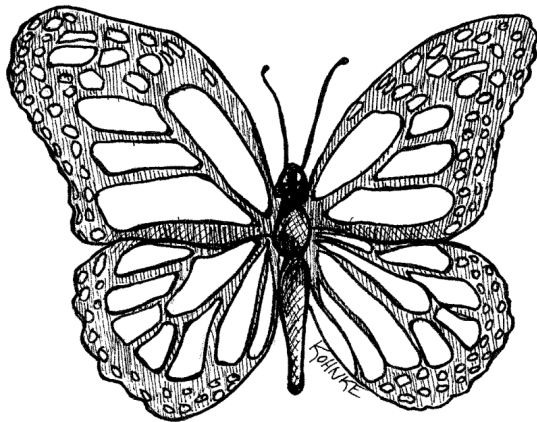
Esta entrega es opcional y está pensada como una “*E3 en esteroides*”. En particular:

- 1 Deben implementar habilidades con nuevos efectos de estado
- 2 Deben implementar más habilidades con efectos especiales

Esta entrega será la última instancia para demostrar que pueden ganarle a los desafíos del proyecto.

Emergence

by Jeff Langr



Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

¿Pero cómo lo logramos?

Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

¿Pero cómo lo logramos?

Idea: Antes de programar, dediquemos mucho tiempo a crear un diseño perfecto

Todos queremos generar software de calidad, con diseños robustos que nos permitan agregarle funcionalidades y adaptarnos a los cambios pedidos por el usuario.

¿Pero cómo lo logramos?

Idea: Antes de programar, dediquemos mucho tiempo a crear un diseño perfecto

¿Alguien ve algún problema con esta idea?

El diseño perfecto depende del problema a resolver y, normalmente, el problema que queremos resolver cambia en el tiempo según las necesidades del usuario/empresa.

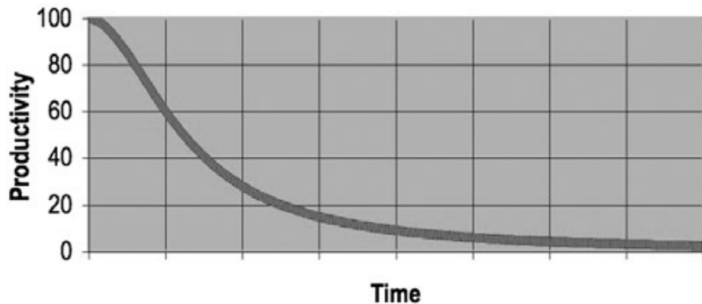
El diseño perfecto depende del problema a resolver y, normalmente, el problema que queremos resolver cambia en el tiempo según las necesidades del usuario/empresa.



Incluso si el problema no cambia (como en el proyecto) es muy difícil que nuestro diseño inicial sea bueno. Sobrepensar nuestro diseño inicial suele ser una mala idea.

Idea 2: Lancémonos a programar con la esperanza de que todo saldrá bien :)

Idea 2: Lancémonos a programar con la esperanza de que todo saldrá bien :)



Spoiler: Todo salió mal.

¿Pero entonces cómo lo hacemos para crear buen software?

¿Pero entonces cómo lo hacemos para crear buen software?

Emergence

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

¿Pero entonces cómo lo hacemos para crear buen software?

Emergence

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

Pregunta para ustedes:

¿Por qué seguir estas reglas nos podría ayudar a mejorar el diseño de nuestro software?

Regla 1: Pasa todos los tests.

Regla 1: Pasa todos los tests.

Lo más importante es que nuestro software funcione como debe. Hoy por hoy, la forma predilecta para verificar el funcionamiento de software es mediante tests automáticos.

¿Pero qué tienen que ver los tests con la calidad del diseño?

Regla 1: Pasa todos los tests.

Lo más importante es que nuestro software funcione como debe. Hoy por hoy, la forma predilecta para verificar el funcionamiento de software es mediante tests automáticos.

¿Pero qué tienen que ver los tests con la calidad del diseño?

Se dan tres argumentos principales:

- 1 Es más fácil testear clases pequeñas que hacen una sola cosa a la vez.
- 2 Clases que están muy acopladas son difíciles de testear.
- 3 Código testeado es más fácil de limpiar.

... por lo que agregar tests nos empujará hacia diseños que sigan buenos principios de diseño.

Regla 2: No tengas código duplicado.

Regla 2: No tengas código duplicado.

“Duplication is the primary enemy of a well-designed system. It represents additional work, additional risk, and additional unnecessary complexity.”

– Jeff Langr.

Regla 2: No tengas código duplicado.

“Duplication is the primary enemy of a well-designed system. It represents additional work, additional risk, and additional unnecessary complexity.”

– Jeff Langr.

“Cada vez que copies y pegues código piensa que probable hay algo mejor que podrías hacer.”

– Matías Recabarren.

```
2 public class VacationPolicy {  
3     public void accrueUSDivisionVacation() {  
4         // code to calculate vacation based on hours worked to date  
5         // ...  
6         // code to ensure vacation meets US minimums  
7         // ...  
8         // code to apply vaction to payroll record  
9         // ...  
10    }  
11    public void accrueEUDivisionVacation() {  
12        // code to calculate vacation based on hours worked to date  
13        // ...  
14        // code to ensure vacation meets EU minimums  
15        // ...  
16        // code to apply vaction to payroll record  
17        // ...  
18    }  
19 }
```

```
2 public abstract class VacationPolicy {
3     public void accrueVacation() {
4         calculateBaseVacationHours();
5         alterForLegalMinimums();
6         applyToPayroll();
7     }
8     private void calculateBaseVacationHours() { /* ... */ };
9     protected abstract void alterForLegalMinimums();
10    private void applyToPayroll() { /* ... */ };
11 }
12 public class USVacationPolicy: VacationPolicy {
13     protected override void alterForLegalMinimums()
14         { /* US specific logic */ }
15 }
16 public class EUVacationPolicy: VacationPolicy {
17     protected override void alterForLegalMinimums()
18         { /* EU specific logic */ }
19 }
```

Regla 3: Que el código deje clara la intención del programador.

Regla 3: Que el código deje clara la intención del programador.

El código se lee más de lo que se escribe. Por lo mismo, ser un profesional implica crear código que sea fácil de entender por el resto de tu equipo (incluyéndote a ti).

¿Cómo creo código que sea fácil de entender por otros?

Regla 3: Que el código deje clara la intención del programador.

El código se lee más de lo que se escribe. Por lo mismo, ser un profesional implica crear código que sea fácil de entender por el resto de tu equipo (incluyéndote a ti).

¿Cómo creo código que sea fácil de entender por otros?

- Usa buenos nombres. El nombre de una clase o método es bueno si al leer su código no te sorprende su contenido.
- Que el código de tus clases y funciones sean cortos.
- Que tus test unitarios sean fáciles de entender (sirven como documentación).
- ...

Regla 3: Que el código deje clara la intención del programador.

El código se lee más de lo que se escribe. Por lo mismo, ser un profesional implica crear código que sea fácil de entender por el resto de tu equipo (incluyéndote a ti).

¿Cómo creo código que sea fácil de entender por otros?

- Usa buenos nombres. El nombre de una clase o método es bueno si al leer su código no te sorprende su contenido.
- Que el código de tus clases y funciones sean cortos.
- Que tus test unitarios sean fáciles de entender (sirven como documentación).
- ...

Pero la regla más importante es que lo **intentas**.

Regla 4: Reduce el número de clases y métodos.

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

A veces, nos encontramos que software cuyo alto número de clases y métodos se debe simplemente a dogmatismos sin sentido.

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

A veces, nos encontramos que software cuyo alto número de clases y métodos se debe simplemente a dogmatismos sin sentido. E.j.,

- Forzar a que se defina una interfaz por cada clase en tu programa.
- Forzar a que se separen datos de comportamiento en clases distintas.
- ...

Regla 4: Reduce el número de clases y métodos.

Esta es la regla con **menos prioridad** pero su rol es recordarnos que no hay que obsesionarse con las buenas prácticas y limpieza de nuestros códigos.

A veces, nos encontramos que software cuyo alto número de clases y métodos se debe simplemente a dogmatismos sin sentido. E.j.,

- Forzar a que se defina una interfaz por cada clase en tu programa.
- Forzar a que se separen datos de comportamiento en clases distintas.
- ...

Entonces, es bueno minimizar el número de clases y métodos en la medida de lo posible... pero no al costo de romper las reglas 1, 2 y 3.

Resumen

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

Resumen

Si sigues estas 4 reglas mientras desarrollas software, un buen diseño emergerá solo:

- 1 Pasa todos los tests.
- 2 No tengas código duplicado.
- 3 Que el código deje clara la intención del programador.
- 4 Reduce el número de clases y métodos.

Espera, estoy confundido.



Espera, estoy confundido.



¿En el curso de diseño me estás recomendando que no haga un diseño?

Capítulo 11: Systems by Dr. Kevin Dean Wampler

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

Building architects have to do BDUF because it is not feasible to make radical architectural changes to a large physical structure once construction is well underway. Although software has its own physics, it is economically feasible to make radical change, if the structure of the software separates its concerns effectively.

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

*Building architects have to do BDUF because it is not feasible to make radical architectural changes to a large physical structure once construction is well underway. Although software has its own physics, it is economically feasible to make radical change, **if the structure of the software separates its concerns effectively.***

Capítulo 11: Systems by Dr. Kevin Dean Wampler

It is not necessary to do a Big Design Up Front (BDUF). In fact, BDUF is even harmful because it inhibits adapting to change, due to the psychological resistance to discarding prior effort and because of the way architecture choices influence subsequent thinking about the design.

*Building architects have to do BDUF because it is not feasible to make radical architectural changes to a large physical structure once construction is well underway. Although software has its own physics, it is economically feasible to make radical change, **if the structure of the software separates its concerns effectively.***

This means we can start a software project with a “naively simple” but nicely decoupled architecture, delivering working user stories quickly, then adding more infrastructure as we scale up.

El extracto clave es el siguiente:

“Although software has its own physics, it is economically feasible to make radical change, if the structure of the software separates its concerns effectively.”

El extracto clave es el siguiente:

“Although software has its own physics, it is economically feasible to make radical change, if the structure of the software separates its concerns effectively.”

¿Cuál es una buena estructura base para comenzar?

Modelo-Vista-Controlador